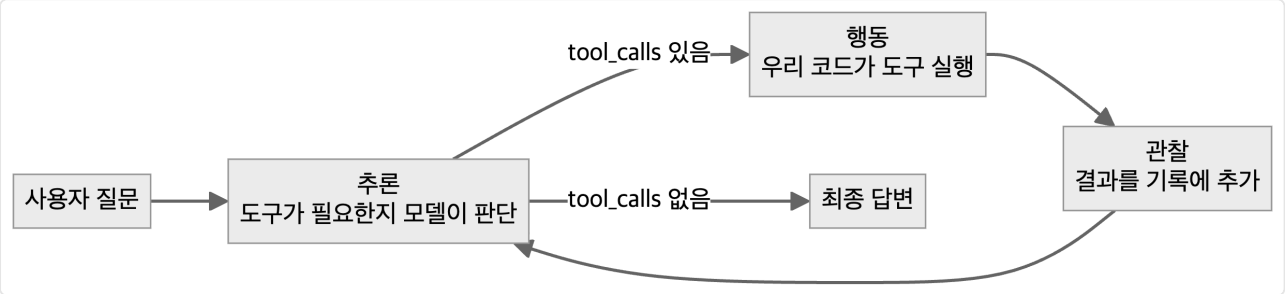
ReAct 수동 루프 구현 (실습)

4-1. ReAct 패턴
4-2. 수동 루프 직접 짜기 (실습)
4-3. 원리 정리

4-1. ReAct 패턴

> 추론과 행동을 번갈아 반복하는 ReAct 패턴의 동작 원리를 이해해봅니다.

ReAct는 2022년 구글 연구진이 발표한 논문에서 나온 패턴입니다. Reasoning and Acting의 약자이고 생각하고 행동하기라는 뜻입니다. 추론(Reasoning)과 행동(Acting)을 번갈아 반복한다는 말입니다. **LLM이 도구를 어떻게 쓸지 스스로 결정하게 하는 방식**입니다.



이것이 왜 강력한가 하면 흐름 제어를 코드가 아니라 LLM이 하기 때문입니다.

"검색 결과가 없으면 키워드를 바꿔 재검색하라" 같은 로직을 if/else 엣지로 짜지 않아도 됩니다. 시스템 프롬프트에 자연어로 적어 두면 LLM이 매 단계 스스로 판단합니다.

도구를 새로 추가할 때도 그래프 구조를 바꾸지 않습니다. 도구 목록에 하나 더하면 끝입니다.

이렇게 할 수 있는 기술적 기반이 1일차의 5번째 세션 5-1에서 본 Tool Calling입니다. LLM은 사용할 수 있는 도구 목록(이름, 설명, 인자 스키마)을 함께 받습니다. 그러면 텍스트 답변 대신 "이 도구를 이 인자로 호출해 달라"는 구조화된 요청(`tool_calls`)을 반환할 수 있습니다.

LLM은 도구를 직접 실행하지 못합니다. 호출 요청서만 씁니다. 실제 실행은 우리 코드나 프레임워크가 하고 그 결과를 다시 LLM에게 보여 주는데 이 왕복이 ReAct 루프의 몸통입니다.

"LLM이 알아서 도구를 실행하는 것 아닌가?"라는 오해가 가장 흔합니다. 아닙니다. LLM은 실행 요청만 만들고 실행과 결과 전달은 전적으로 우리 몫입니다. 4-2의 수동 루프가 바로 이 우리 몫을 전부 드러내 보여 줍니다.

> LLM은 호출 요청서만 쓰고 실행은 우리 코드가 한다는 ReAct의 역할 분담을 확인했습니다.

4-2. 수동 루프 직접 짜기 (실습)

> while 루프로 도구 선택, 실행, 결과 관찰, 재판단을 직접 구현해봅니다.

프레임워크의 도움 없이 while 루프로 ReAct Agent를 손으로 짜 봅니다.

손으로 한 번 짜 보면 5번째 세션에서 LangGraph가 무엇을 대신해 주는지가 정확히 보입니다. 지금은 원리를 눈으로 보는 중이므로 이 코드를 외우거나 매번 짤 필요는 없습니다.

따라하기

1. 전체 코드를 `day3_06_manual_react.py` 로 저장해 실행합니다.

```
# day3_06_manual_react.py - ReAct 루프를 while 루프로 직접 구현
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


# ---------- 1) 도구 정의 ----------
@tool
def search_employee(name: str) -> str:
    """임직원 이름으로 소속 팀과 이메일을 조회합니다. 임직원 정보 질문에 사용하세요."""
    # 교육용 더미 데이터입니다. 실제 사내 데이터와 무관합니다.
    employees = {
        "김하늘": "김하늘 / 클라우드운영팀 / haneul.kim@samsungsds.example.com",
        "박도윤": "박도윤 / 물류플랫폼팀 / doyun.park@samsungsds.example.com",
    }
    return employees.get(name, f"'{name}' 님을 찾을 수 없습니다.")


@tool
def calculate(expression: str) -> str:
    """수식 문자열을 계산해 정확한 결과를 반환합니다. 예: '34 * 27', '(15 - 5) / 2'"""
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"계산 실패: {e}"


tools = [search_employee, calculate]
tools_by_name = {t.name: t for t in tools}   # 이름으로 도구를 찾기 위한 사전
```

```
# ---------- 2) 도구 목록을 모델에 알려주기 ----------
llm_with_tools = llm.bind_tools(tools)

# ---------- 3) 대화 기록 초기화 ----------
messages = [
    SystemMessage(content="너는 삼성SDS 사내 어시스턴트야. 필요하면 도구를 사용해."),
    HumanMessage(content="김하늘 님과 박도윤 님이 각각 어느 팀 소속인지 알려주고, 34 곱하기
27도 계산해줘."),
]

# ---------- 4) ReAct 루프: 도구 선택 -> 실행 -> 결과 관찰 -> 재판단 ----------
MAX_TURNS = 10                                      # 안전장치
turn = 0
while turn < MAX_TURNS:
    turn += 1
    ai_message = llm_with_tools.invoke(messages)   # (추론) 모델이 도구 필요 여부를
판단
    messages.append(ai_message)                     # 응답을 기록에 추가

    if not ai_message.tool_calls:                   # 도구 요청이 없으면 최종 답변
        print(f"\n[최종 답변] {get_text(ai_message)}")
        break

    for tool_call in ai_message.tool_calls:         # (행동) 요청된 도구를 전부 실행
        tool_fn = tools_by_name[tool_call["name"]]
        result = tool_fn.invoke(tool_call["args"])
        print(f"[턴 {turn}] {tool_call['name']}({tool_call['args']}) -> {resu
lt}")

        messages.append(                            # (관찰) 결과를 기록에 추가
            ToolMessage(content=str(result), tool_call_id=tool_call["id"])
        )
    # while 처음으로 돌아감 -> 모델이 도구 결과를 보고 다시 판단 (재판단)
else:
    print("\n[중단] 최대 턴 수를 초과했습니다.")
```

2. 실행 결과 예시는 다음과 같습니다. 모델에 따라 턴 1에서 조회 두 건을 한 번에 요청할 수도 있고 나눠 요청할 수도 있습니다. 이렇게 매번 조금씩 달라지는 경로가 바로 LLM이 흐름을 제어한다는 증거입니다.

```
[턴 1] search_employee({'name': '김하늘'}) -> 김하늘 / 클라우드운영팀 / haneul.kim@s
amsungsds.example.com
[턴 1] search_employee({'name': '박도윤'}) -> 박도윤 / 물류플랫폼팀 / doyun.park@sa
msungsds.example.com
[턴 1] calculate({'expression': '34 * 27'}) -> 918
[최종 답변] 김하늘 님은 클라우드운영팀, 박도윤 님은 물류플랫폼팀 소속입니다. 34 곱하기 27은 918입
니다.
```

3. 루프의 핵심 줄을 읽습니다.

- `@tool` 데코레이터는 일반 파이썬 함수를 LangChain 도구 객체로 바꿉니다. 함수 이름이 도구 이름이 되고 docstring이 도구 설명이 되며 타입 힌트가 인자 스키마가 됩니다.
  LLM은 이 세 가지만 보고 도구를 고르므로 docstring을 대충 쓰면 도구 선택 정확도가 바로 떨어집니다.
  (4일차에서 깊이 다룹니다.)
- `messages` 리스트가 이 루프의 State 역할을 합니다. LLM은 상태가 없는(stateless) API이므로 매 호출마다 지금까지의 기록 전체를 다시 보내야 문맥이 이어집니다. 이 기록 관리가 수동 루프에서 우리가 직접 해야 하는 핵심 잡무입니다.
- `if not ai_message.tool_calls`가 종료 판정입니다. `tool_calls`가 비어 있다는 것은 더 필요한 도구가 없고 content에 최종 답변을 적었다는 뜻입니다.
- `ToolMessage(content=..., tool_call_id=tool_call["id"])`에서 `tool_call_id`는 이 결과가 어느 요청에 대한 응답인지 짝지어 주는 값입니다. 반드시 요청의 id를 그대로 넣어야 합니다.
- 루프 한 바퀴가 정확히 추론, 행동, 관찰이고 바퀴를 다시 도는 것 자체가 재판단입니다.

막힐 때

- "tool_calls가 있는 메시지 다음에는 tool 메시지가 와야 한다"는 400 에러가 납니다.
  가장 흔한 에러입니다. 모델이 요청한 tool_call마다 ToolMessage를 하나씩 만들어 넣어야 합니다.
  `tool_call_id`도 정확히 일치해야 합니다.
- ToolMessage만 넣고 그 앞의 AIMessage를 기록에 안 넣어도 형식 에러가 납니다.
  요청서와 결과서는 항상 짝으로 기록에 남긴다고 기억하면 됩니다.
- `tool_call["args"]`를 JSON 파싱하려다 에러가 납니다.
  이미 dict입니다. 그대로 `tool_fn.invoke(...)`에 넘기면 됩니다.
- 모델이 도구를 안 쓰고 지어내서 답합니다.
  시스템 프롬프트에 "임직원 정보는 반드시 search_employee로 조회해"처럼 사용 조건을 명시하면 개선됩니다. 도구 docstring을 구체적으로 쓰는 것도 효과가 큽니다.

확인: 도구 호출 로그가 순서대로 찍히고 최종 답변에 두 사람의 팀과 계산 결과 918이 모두 들어 있으면 성공입니다.

> ReAct 루프의 기록 관리, 종료 판정, 도구 실행이 전부 우리 코드의 몫이라는 걸 직접 구현하며 확인했습니다.

4-3. 원리 정리

> 손으로 짠 루프의 각 부분이 LangGraph의 어떤 부품에 대응하는지 정리해봅니다.

방금 짠 수동 루프에서 우리가 직접 관리한 것은 네 가지입니다.

- **기록 관리**: 대화 기록 리스트를 만들고 매 단계 새 메시지를 추가했습니다.
- **종료 판정**: tool_calls 유무를 검사해 계속할지 끝낼지를 갈랐습니다.
- **도구 실행**: 이름으로 도구를 찾아 실행하고 결과를 ToolMessage로 포장했습니다.
- **루프 돌리기**: while 루프를 직접 돌리고 최대 턴 수 안전장치도 붙였습니다.

이를 LangGraph 개념에 대응시키면 정확히 이렇게 됩니다.

수동 루프에서 우리가 한 일	LangGraph에서 대신하는 것
messages 리스트 관리와 추가	State + `add_messages` reducer
tool_calls 유무로 계속/종료 판정	조건부 엣지 (라우팅 함수)
도구 찾기, 실행, ToolMessage 포장	`ToolNode` (내장 노드)
while 루프 자체	Cycle (tools에서 agent로 돌아가는 엣지)
MAX_TURNS 안전장치	`recursion_limit`

이 표가 3일차 전체의 요약입니다. 첫 번째 세션부터 3번째 세션까지 배운 State, reducer, 조건부 엣지, Cycle이 전부 수동 ReAct 루프의 잡무를 프레임워크로 옮기기 위한 부품이었던 것입니다.

수동 루프도 잘 동작하는데 왜 LangGraph로 옮길까요. 실무 관점의 이유는 세 가지입니다.

- **중단과 재개**: 도구 실행 전에 사람 승인을 받으려면(5일차 HITL) 루프를 멈췄다 다시 시작할 수 있어야 합니다. while 루프로는 매우 어렵습니다. 대신 LangGraph는 체크포인트(그때까지의 State를 저장해 둔 지점)를 두고 아무 지점에서나 재개합니다.
- **관측과 디버깅**: 그래프 구조는 시각화되고 각 노드 실행이 추적(trace)됩니다. while 루프 안의 진행 상황은 print를 심지 않으면 보이지 않습니다.
- **확장**: 병렬 도구 실행, 스트리밍(다 끝나길 기다리지 않고 결과를 나오는 대로 받기), 서브그래프(6일차 Multi-Agent) 같은 기능이 그래프 위에 얹혀 있습니다.

실무에서는 LangGraph가 이 루프를 대신 처리합니다. 다만 원리를 손으로 짜 봤기 때문에 이제 LangGraph 코드를 볼 때 이 노드가 내 루프의 어느 줄에 해당하는지 알고 읽을 수 있습니다.

원리는 while 루프이고 실무는 LangGraph가 대신한다는 대응 관계를 확인했습니다.

▼ ReAct 논문이 실제로 바꾼 것

ReAct 논문이 나온 2022년에는 Chain-of-Thought가 한창 주목받고 있었습니다. 단계별로 생각해 보라고 시키면 추론이 좋아진다는 발견이었죠.

문제는 생각만 잘한다는 거였습니다. 아무리 추론이 좋아도 모델 안에 없는 최신 정보나 사내 데이터에는 손이 닿지 않으니 그럴듯한 오답이 나왔습니다.

ReAct가 한 일은 간단합니다. 추론과 행동을 번갈아 시키고, 행동한 결과를 다시 대화 기록에 넣어 다음 추론의 재료로 쓰게 했습니다. 4-2에서 손으로 짠 루프가 그대로 이 구조입니다.

당시엔 이걸 전부 문자열 파싱으로 했습니다. 모델이 Action: search[...] 같은 형식을 텍스트로 뱉으면 코드가 정규식으로 잘라 실행했고, 형식이 조금만 어긋나도 깨졌습니다.

지금은 모델이 tool_calls라는 구조화된 필드로 돌려줍니다. 4-2 코드에 파싱 로직이 한 줄도 없던 게 그래서입니다. 논문의 아이디어는 그대로 남았고, 바뀐 쪽은 모델이 규격을 지키게 됐다는 점입니다.